

TLS



14.1 Introduction

14.2 TLS Record Protocol

14.3 TLS Handshake Protocol

14.4 Summary

14.1 Introduction

- ❑ Transport Layer Security (TLS) [RFC2246]
 - ❑ TLS provides transport layer security for Internet applications
 - ❑ It provides for confidentiality and data integrity over a connection between two end points
 - ❑ TLS operates on a reliable transport, such as TCP, and is itself layered into
 - TLS Record Protocol
 - TLS Handshake Protocol
-

□ Advantage of TLS

- applications can use it transparently to securely communicate with each other
- TLS is visible to applications, making them aware of the cipher suites and authentication certificates negotiated during the set-up phases of a TLS session

14.2 TLS Record Protocol

- ❑ TLS Record Protocol layers on top of a reliable connection-oriented transport, such as TCP
 - ❑ TLS Record Protocol
 - provides data confidentiality using symmetric key cryptography
 - provides data integrity using a keyed message authentication checksum (MAC)
 - ❑ The keys are generated uniquely for each session based on the security parameters agreed during the TLS handshake
-

□ Basic operation of the TLS Record Protocol

1. read messages for transmit
 2. fragment messages into manageable chunks of data
 3. compress the data, if compression is required and enabled
 4. calculate a MAC
 5. encrypt the data
 6. transmit the resulting data to the peer
-

-
- At the opposite end of the TLS connection, the basic operation of the sender is replicated, but in the reverse order
 1. read received data from the peer
 2. decrypt the data
 3. verify the MAC
 4. decompress the data, if compression is required and enabled
 5. reassemble the message fragments
 6. deliver the message to upper protocol layers
-

14.3 TLS Handshake Protocol

- ❑ TLS Handshake Protocol is layered on top of the TLS Record Protocol
- ❑ TLS Handshake Protocol is used to
 - authenticate the client and the server
 - exchange cryptographic keys
 - negotiate the used encryption and data integrity algorithms before the applications start to communicate with each other

-
- Figure 14.1 illustrates the actual handshake message flow
 - [Step1]
 - the client and server exchange Hello messages
 - the client sends a **ClientHello** message, which is followed by the server sending a **ServerHello** message
-

-
- ❑ these two messages establish the TLS protocol version, the compression mechanism used, the cipher suite used, and possibly the TLS session ID
 - ❑ additionally, both a random client nonce and a random server nonce are exchanged that are used in the handshake later on
-

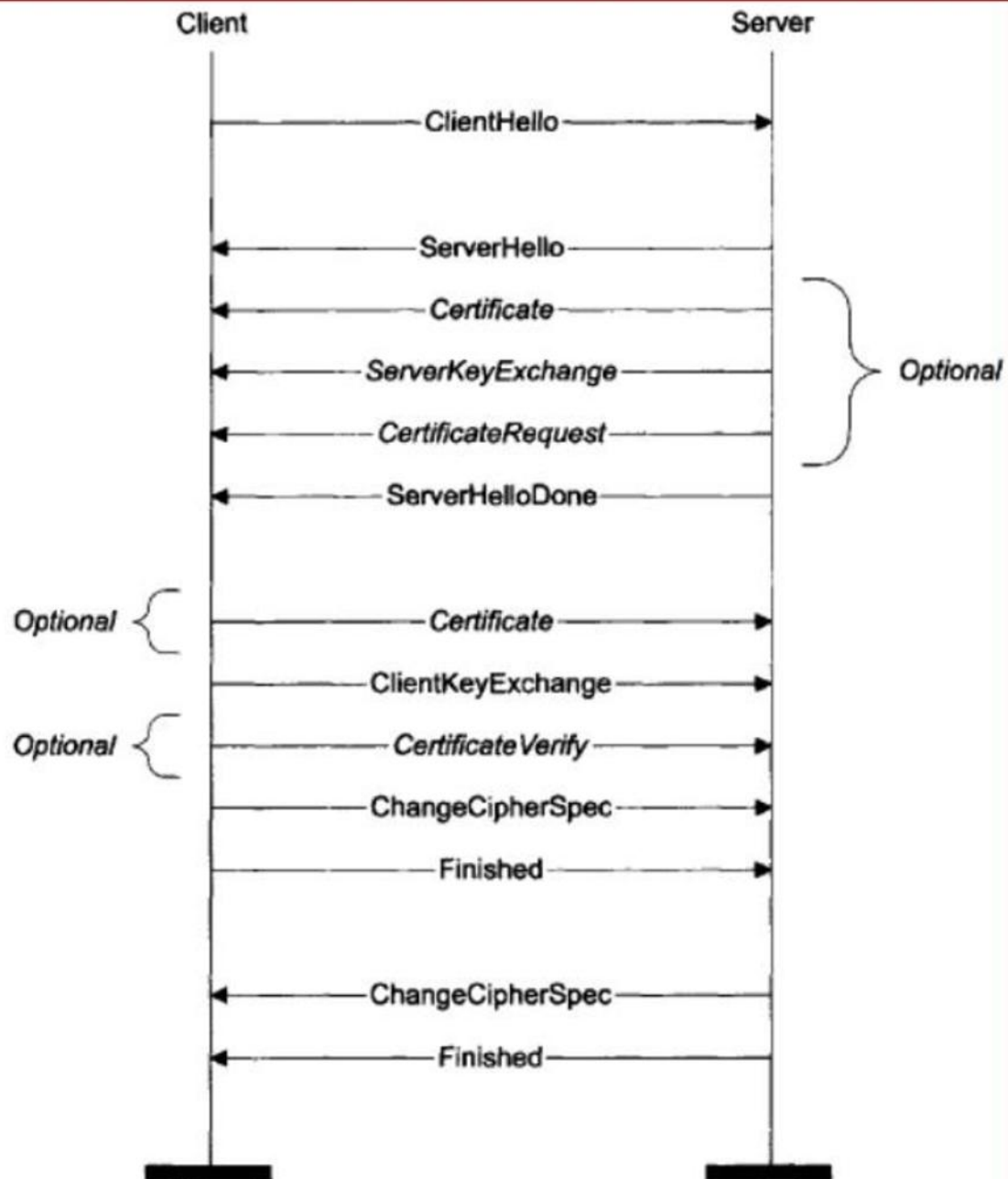


Figure 14.1 The TLS handshake.

■ [Step2]

- the server may send any messages associated with the ServerHello
- depending on the selected **cipher suite**, it will send its certificate for authentication
- the server may also send a **key exchange message** and a **certificate request message** to the client, depending on the selected cipher suite

-
- to mark the end of the ServerHello and the Hello message exchange, the server sends a **ServerHelloDone** message

□ [Step3]

- next, if requested, the client will send its **certificate** to the server
 - in any case, the client will then send a **key exchange message** that sets the pre-master secret between the client and the server
 - optionally, the client may also send a **Certificate Verify message** to explicitly verify the certificate that the server requested
-

□ [Step4]

- then, both the client and the server send the **ChangeCipherSpec messages** and enable the newly negotiated cipher spec
 - the first message passed in each direction using the new algorithms, keys and secrets is the **Finished message**, which includes a digest of all the handshake messages
 - each end inspects the Finished message to verify that the handshake was not tampered with
-

註：

- Digest of all the handshake messages
 - means the results of applying a one-way hash function to the handshake messages
-

14.4 Summary

- ❑ TLS protocol provides transport layer security for Internet applications and confidentiality using symmetric key cryptography and data integrity using a keyed MAC
 - ❑ It also includes functionality for client and server authentication using public key cryptography
-